3D-PRINTABLE · ESP32-C3 SUPER MINI · MQTT

WindGauge

stepper edition

A 205 mm wall-mounted dial that shows live wind direction from your weather station. A 28BYJ-48 stepper turns the pointer, a hall sensor finds North on every start-up, and an LED ring lights the dial from behind.



Prints on a 220 mm-class bed. Largest part 204.8 mm (bezel). Five parts, no supports.	Self-homing. Hall sensor and magnet set North, and the sketch measures steps per revolution itself.
Screw-on bezel. M200 × 4 thread with print clearance, 190 mm acrylic window.	Clip-in parts. Dial, LED strip, boards and breadboard all clip in. One screw hangs it on the wall.

Design and build: Andy Hudson-Smith · digitalurban.org

HOW IT WORKS

From a weather feed to a moving pointer

Your weather station publishes the wind direction (in degrees) to an MQTT broker. The ESP32-C3 Super Mini subscribes over Wi-Fi and turns the 28BYJ-48 stepper by the shortest path to that bearing. The pointer sits on a small printed **magnet carrier** on the motor shaft. A 5 mm magnet in the carrier passes over a KY-003 hall sensor at North, so the gauge can find North on start-up and re-zero itself every six hours.

A WS2812B strip clips into a channel under the dial, above all the boards and wiring, so it lights the dial evenly without casting wire shadows.



Exploded view, back to front: case with motor, ULN2003, breadboard + ESP32-C3 and LED channel; magnet carrier; dial; pointer; acrylic window; screw-on bezel.

INSIDE THE CASE

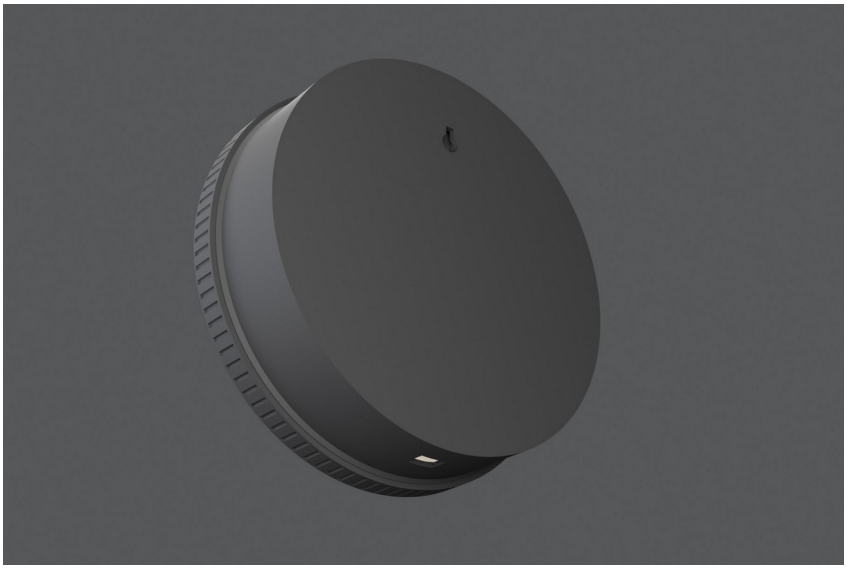
Where everything goes

Every part has its own printed seat, so nothing needs glue except the magnet. The 28BYJ-48 sits in the centre block with the magnet carrier on its shaft. The KY-003 hall sensor is in the cradle above it, under the magnet's path. The ULN2003 driver snaps in to the right, and the mini breadboard with the ESP32-C3 Super Mini sits in a tray at the bottom: its side tabs drop into two notches and the adhesive back holds it down. The board's USB-C end faces a 14 × 10 mm opening in the bottom wall.

The LED strip clips into a channel round the inside wall, just under the dial, so it sits above every board and wire. A gap in the channel's lip at 5 o'clock lets the strip's wires drop down to the breadboard. On the back, a single centred keyhole hangs the gauge on one screw.



From the front, dial removed.



From below and behind: keyhole and USB opening.

WHAT YOU NEED

Parts list

Printed parts

Part	File	Print orientation	Size (mm)
Case	WG-01_Case_back-down	Back down	199.7 × 199.7 × 53.3
Dial: raised (choose one)	WG-02_Dial_face-up_M600	Face up, colour change at 2.2 mm	190.4 × 3.0
Dial: flat two-colour (choose one)	WG-02F_Flat_dial_2colour_Bambu (.3mf)	Face down, AMS or manual swaps	190.4 × 2.0
Bezel	WG-03_Bezel_front-down	Front down	204.8 × 15.3
Pointer	WG-04_Pointer	Underside down	12 × 117 × 4.4
Magnet carrier	WG-05_Magnet_carrier	Underside down	12 × 32.5 × 17.2

The bezel needs at least 205 mm in one direction. It fits a Prusa MK4 (250 × 210) and Bambu X1, P1 and A1 (256 × 256), but not an A1 mini or Prusa MINI.

Bought parts

Item	Notes
ESP32-C3 Super Mini (with headers)	Wi-Fi and native USB-C. 3.3 V logic. The same board as the Open Gauges pressure and temperature gauges.
Mini breadboard, 170 tie-points (ELEGOO or similar)	About 46 × 35 × 9.4 mm. Its side tabs drop into the tray notches; peel the adhesive back to stick it down.
28BYJ-48 stepper (5 V) + ULN2003 driver board	The 5 V version, not the 12 V one.
KY-003 hall sensor module (A3144)	Powered from 5 V.
Neodymium magnet, 5 mm diameter, up to 3 mm thick	Glued into the magnet carrier.
WS2812B LED strip, 10 mm wide, up to 2.2 mm thick	About 0.6 m round the inside of the case. Bare or IP30 strip, not the thick silicone-sleeved type.
Clear acrylic disc, Ø190 × 3 mm	Laser cut from Acrylic_190mm_x3_disc.dxf, or cut from 3 mm A4 sheet and sand to 189.5–190.2 mm.
2 × M3 × 6 pan-head screws	Motor ears; they self-tap into the printed pilot holes.
Jumper wires, USB-C cable, 5 V USB supply	A 2 A supply gives the LEDs headroom.
1 × wall screw	Head up to 9.5 mm across and 3.5 mm thick.

PRINTING

Print settings

Setting	Value
Material	PLA (black case, bezel and carrier; white dial; red pointer looks good)
Layer height	0.2 mm
Walls / infill	3 walls, 15 % infill
Supports	None: every part is already oriented in its file
Brim	Not needed. Clean the sheet with IPA for the large parts.

Part notes

Case. The LED channel's top cap is a short 2.8 mm flat overhang off the wall; it prints without supports but may sag slightly underneath. The USB opening and keyhole are bridged.

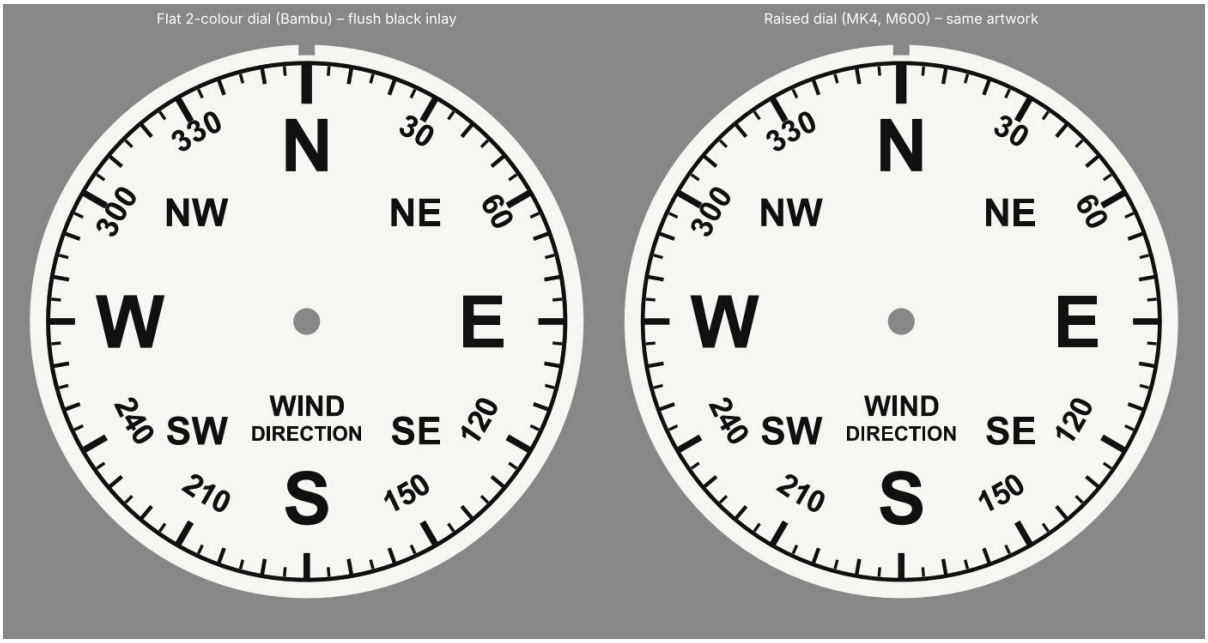
Bezel. The M200 × 4 thread has 0.2 mm clearance built in. Print it in the same filament as the case so the shrinkage matches.

Pointer and carrier. Both are designed to push on until they stop: the carrier stops on the end of the motor shaft, the pointer on the top of the spigot. That keeps the pointer 1 mm above the dial and 1 mm below the acrylic.

Dial options

Raised dial (single-extruder). Print WG-02 face up in white and add a filament change (M600) at the 2.2 mm layer: the marks stand 1 mm proud in black.

Flat two-colour dial. WG-02F has the black marks inlaid flush, 0.6 mm deep, and prints face down so the face is dead flat. On a Bambu with AMS, open the .3mf and set *Dial white* to white and *Marks black* to black. On a Prusa MK4 without an MMU you can do it with three or four manual swaps: set up a two-extruder "Single Extruder Multi Material" profile with `M600` as the tool-change G-code and a small wipe tower.



The two dials share the same artwork. Left: flat two-colour inlay. Right: raised marks.

ASSEMBLY

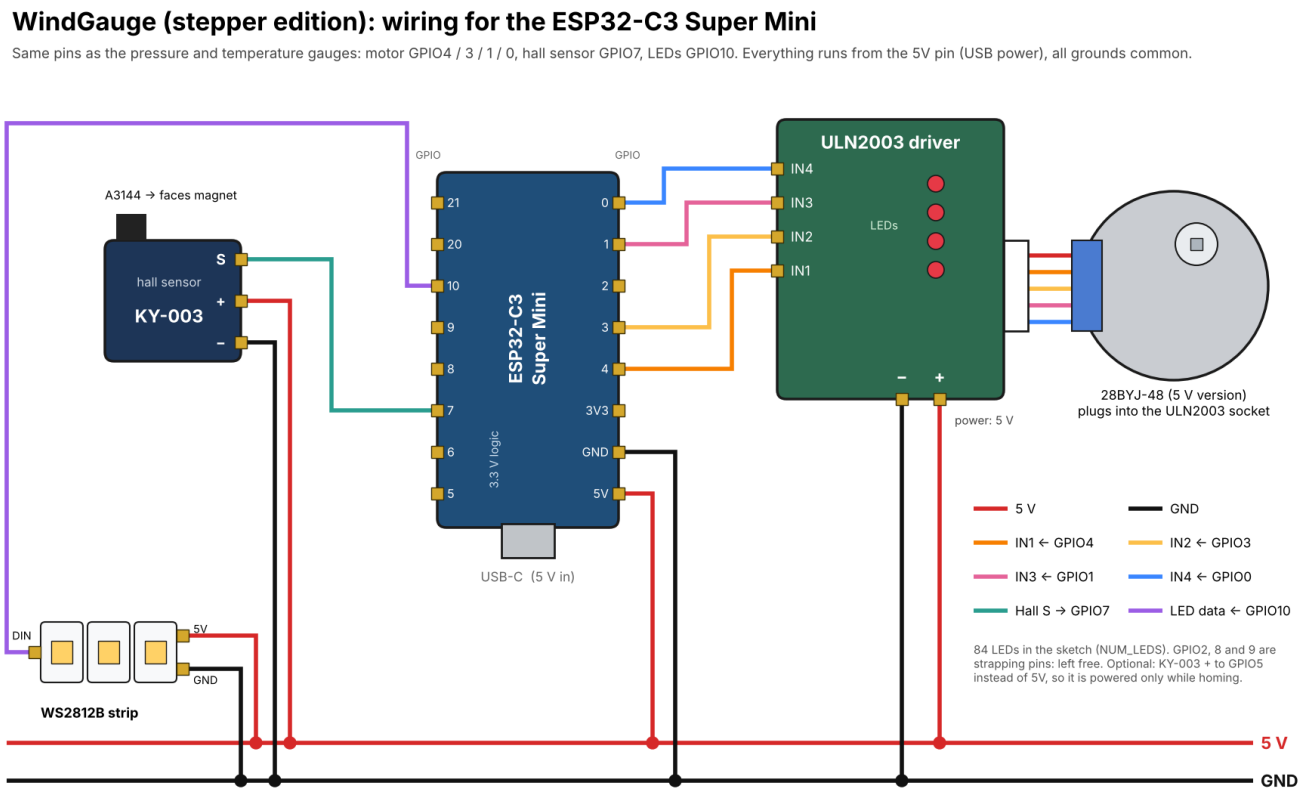
Putting it together

Test the electronics on the bench before you close the case: it is much easier to fix a wire with the dial out.

1. **Test-fit first.** Before adding electronics, check the dial clicks into the case and the bezel screws on.
2. **Motor.** Drop the 28BYJ-48 into its pocket from the front, wire cover pointing down, and fix the ears with two M3 × 6 screws.
3. **Driver.** Snap the ULN2003 board into its cradle to the right of the motor and plug in the motor cable.
4. **Breadboard and ESP32-C3.** Press the mini breadboard into the tray at the bottom: its side tabs go into the two notches. Peel the adhesive backing first to stick it down. Plug the Super Mini in at the wall end of the breadboard, USB-C end against the case wall, then check a cable reaches through the opening.
5. **Hall sensor.** Bend the A3144 so it lies flat, face up, on the small shelf below the KY-003 cradle, and snap the board in.
6. **Magnet carrier.** Power up and hold the magnet over the sensor: the KY-003 LED should light. If not, flip the magnet. Glue it into the carrier pocket the right way up, then push the carrier onto the shaft until it stops.
7. **LED strip.** Starting at the wire gap at 5 o’clock, drop the strip’s lower edge behind the lip and press the top edge under the cap, LEDs facing inward. Work round the case; no adhesive needed.
8. **Wire up** as shown on the next page, upload the sketch and let it home and sweep to South and back.
9. **Dial.** Line up the notch with the rib at 12 o’clock, tuck it under two of the snap bumps and press the third side down until it clicks.
10. **Pointer.** Push it onto the spigot until it stops. It only fits one way.
11. **Window and bezel.** Peel the film off the acrylic, clip it into the bezel from behind, and screw the bezel on hand-tight until it seats on the case rim.
12. **Hang it.** The single keyhole puts the screw 67 mm directly above the dial centre. Leave the screw head about 3 mm out from the wall.

ELECTRONICS

Wiring the ESP32-C3 Super Mini



ESP32-C3 Super Mini	Connects to
GPIO4 / GPIO3 / GPIO1 / GPIO0	ULN2003 IN1 / IN2 / IN3 / IN4
GPIO7	KY-003 S (signal)
GPIO10	WS2812B DIN (data in)
5V	ULN2003 +, KY-003 +, LED strip 5V
GND	ULN2003 −, KY-003 −, LED strip GND

As built, no extra components are needed: the KY-003 and the strip connect straight to the pins. Two optional extras if you want extra margin: the KY-003 board pulls its output up to 5 V while the ESP32-C3's pins are rated for 3.3 V, so a 10 kΩ / 20 kΩ divider on S keeps it within spec; and a 330 Ω resistor on LED data plus a 1000 µF capacitor across the strip's power is standard WS2812B practice. GPIO2, GPIO8 and GPIO9 are strapping pins on the C3, so they are left free. If you want the hall sensor to draw nothing between homings, wire KY-003 + to GPIO5 instead of 5V: the sketch switches it on only while homing.

FIRMWARE

The Arduino sketch

The full sketch is printed at the end of this guide and included as `WindGauge_ESP32C3.ino`.

Set up

1. In the Arduino IDE, install the board package **esp32 by Espressif Systems (v3.x)** and select **ESP32C3 Dev Module**. In the Tools menu, set USB CDC On Boot to Enabled for the Serial Monitor.
2. Install the libraries **AccelStepper**, **ArduinoJson (v7)**, **PubSubClient** and **Adafruit NeoPixel** from the Library Manager.
3. Edit the settings at the top of the sketch (below), then upload with the Serial Monitor open at 115200 baud.

What to change

Setting	What it does
`ssid`, `password`	Your Wi-Fi network.
`mqtt_server`, `mqtt_port`	Your MQTT broker.
`wind_topic`	The topic your station publishes wind direction to. The payload is a plain number of degrees, e.g. `247`. A weeWX loop packet (JSON with a windDir key) also works.
`NUM_LEDS`	Number of LEDs on your strip (84 in the sketch).
`LED_BRIGHTNESS`	Backlight brightness, 0–255 (50 in the sketch). Turn it down if the board resets or the USB supply struggles.
`northOffsetDeg`	Fine-tunes North (see below).
`backlashDeg`	Gearbox play taken up on anticlockwise moves (typically 2–6°). Every move finishes clockwise, so the pointer always lands from the same side.

What happens at start-up

The LED ring shows how far start-up has got: red when the sketch starts, blue while it connects to Wi-Fi, then yellow while the motor turns until the magnet reaches the hall sensor and makes one full turn to measure its real steps per revolution, so any 28BYJ-48 calibrates itself. It then moves to true North, and the ring turns purple while it sweeps to South and back as a self-test, then white when it is ready. Wi-Fi connects before the motor runs, so the two biggest power draws never overlap. From then on it follows every new reading by the shortest route, ignores changes under 2°, switches the motor coils off between moves, and re-homes on the magnet every six hours to cancel any drift.

Setting North exactly

Watch the pointer when the Serial Monitor prints *“At North. Check the pointer against the N mark.”* If it stops short of N (anticlockwise of it), increase `northOffsetDeg`; if it overshoots, decrease it. Negative values are fine. If the sensor reads HIGH on the magnet rather than LOW, change `atMagnet()` in the sketch.

WHEN THINGS GO WRONG

Troubleshooting

Symptom	Likely cause and fix
Motor keeps turning at start-up and never stops	The hall sensor isn't triggering. Check the KY-003 LED lights with the magnet over it; flip the magnet if not. The gap should be about 1.5 mm.
Ring stays red, blue or dark	Red or dark: the sketch isn't running. Unplug the USB and plug it back in (after an upload the C3 can stay in download mode). Blue: Wi-Fi isn't connecting. Check the name and password and that it is a 2.4 GHz network; transmit power is set low for the Super Mini's small antenna, so try it closer to the router.
Motor buzzes but doesn't turn	IN1–IN4 swapped. Wire GPIO4, GPIO3, GPIO1 and GPIO0 to IN1–IN4 in that order; the unusual pin order in the AccelStepper line is correct for the 28BYJ-48.
Board resets when the LEDs light	Too much current for USB. Lower `LED_BRIGHTNESS` or use a stronger supply.
Pointer rubs the dial or acrylic	Push the carrier fully onto the shaft and the pointer fully onto the spigot; both have hard stops.
Breadboard won't drop into the tray	Its side tabs don't line up with the notches. The notch width is a parameter (`bb_notch_w`) in the Fusion model.

Files in the download

File	What it is
WG-01 … WG-05 (.stl / .3mf)	The printed parts, already oriented for printing.
WG-02F_Flat_dial_2colour_Bambu.3mf	Flat two-colour dial with both parts in place (white body, black inlay).
WG-02F_…_WHITE / _BLACK_face-down.stl	The same two dial parts as separate STLs for other slicers.
Acrylic_190mm_x3_disc.dxf	Cutting file for the acrylic window.
STEP files and the Fusion .f3d	The full parametric model. Breadboard tray, notches and clearances are user parameters.
WindGauge_ESP32C3.ino	The Arduino sketch.
Wiring diagram (.png / .pdf)	The diagram from this guide at full size.

APPENDIX

Full sketch: WindGauge_ESP32C3.ino

```
/*
  Open Gauges — Wind Direction gauge
  Board: ESP32-C3 Super Mini (Arduino core for ESP32 v3.x)
  Motor: 28BYJ-48 + ULN2003 driver   Homing: KY-003 hall sensor + 5 mm magnet
  LEDs:  WS2812 / NeoPixel strip around the rim (white backlight)

  - Reads wind direction (degrees) from MQTT: a plain number (e.g. ".../windDir" -> 247)
    or a weeWX loop packet (JSON, "windDir" key). Calm readings (no direction) are ignored.
  - Full 360° dial: the pointer always takes the shortest way round.
  - Start-up progress on the ring: RED started, BLUE Wi-Fi, YELLOW homing,
    PURPLE self-test, WHITE ready.
  - Same wiring as the Open Gauges pressure and temperature gauges.
  - KY-003 "+" can go to 5V (simplest), or to GPIO5 so the sensor is powered only
    while homing and draws nothing the rest of the time. The sketch works either way.

  Libraries (Library Manager): AccelStepper, PubSubClient, ArduinoJson (v7), Adafruit NeoPixel
  Tools menu: Board "ESP32C3 Dev Module", USB CDC On Boot "Enabled" (for Serial Monitor)
*/

#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <AccelStepper.h>
#include <Adafruit_NeoPixel.h>

// ============================================================================
// WI-FI / MQTT
// ============================================================================
const char* ssid        = "YOUR_WIFI_NAME";
const char* password    = "YOUR_WIFI_PASSWORD";

const char* mqtt_server = "mqtt.cetools.org";
const int   mqtt_port   = 1883;

// A topic carrying plain degrees (e.g. 247), or a weeWX loop packet (JSON) with "windDir".
const char* wind_topic  = "personal/ucfnaps/downhamweather/windDir";   // edit for your station

// ============================================================================
// PINS — ESP32-C3 Super Mini (avoid strapping pins GPIO2, GPIO8, GPIO9)
// ============================================================================
#define IN1_PIN   4     // ULN2003 IN1
#define IN2_PIN   3     // ULN2003 IN2
#define IN3_PIN   1     // ULN2003 IN3
#define IN4_PIN   0     // ULN2003 IN4
#define HALL_PIN  7     // KY-003 S
#define HALL_PWR  5     // optional: KY-003 + here to power it only while homing
#define LED_PIN   10    // LED strip DIN

// ============================================================================
// DIAL
// ============================================================================
// How far the pointer must travel FORWARD (clockwise) from the magnet trigger
// point to sit exactly on the N mark.
//   Pointer stops short of N (anticlockwise of it) -> increase
//   Pointer overshoots N (clockwise of it)          -> decrease
// Negative values are fine if the magnet triggers after North.
const float northOffsetDeg = 6.0;

// BACKLASH: the 28BYJ-48 gearbox has a few degrees of play. Homing approaches
// clockwise, so clockwise moves land accurately but anticlockwise moves stop short.
// Fix: every anticlockwise move overshoots by this much, then comes back clockwise,
// so the pointer always arrives from the same side.
//   Still short after anticlockwise moves -> increase   (typical 2-6 deg)
```

```
const float backlashDeg = 4.0;

// Don't nudge the pointer for changes smaller than this (degrees)
const float moveDeadbandDeg = 2.0;

// ============================================================================
// LEDS
// ============================================================================
#define NUM_LEDS        84
#define LED_BRIGHTNESS  50           // 0-255; turn it down if the board resets
const uint8_t backlightRGB[3] = {255, 255, 255};

// ============================================================================
// MOTOR
// ============================================================================
long  stepsPerRevolution = 4096;                      // overwritten by calibration
float stepsPerDegree     = stepsPerRevolution / 360.0;
const float homingSpeed  = 300.0;
const float runSpeed     = 400.0;
const float runAccel     = 150.0;
const bool  releaseCoilsWhenIdle = true;   // motor stays cool; gearbox holds the pointer
const unsigned long rehomeIntervalMs = 6UL * 60UL * 60UL * 1000UL;  // re-zero every 6 h

AccelStepper stepper(AccelStepper::HALF4WIRE, IN1_PIN, IN3_PIN, IN2_PIN, IN4_PIN);
Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
WiFiClient   espClient;
PubSubClient client(espClient);

// ============================================================================
// STATE
// ============================================================================
float currentDir = NAN;     // latest wind direction (0-360)
float shownDir   = NAN;     // direction the pointer was last sent to
bool  coilsOn    = true;
unsigned long lastRehomeMs = 0;

// ============================================================================
// HELPERS
// ============================================================================
bool atMagnet() { return digitalRead(HALL_PIN) == LOW; }   // KY-003 pulls LOW on magnet

// The hall sensor (and its on-board LED) is only powered while homing when wired to GPIO5.
// When off, the signal pin is left as plain INPUT so the internal pull-up can't leak
// current back through the module's own pull-up resistor.
void hallPower(bool on) {
  if (on) {
    digitalWrite(HALL_PWR, HIGH);
    pinMode(HALL_PIN, INPUT_PULLUP);
    delay(20);                                // let the sensor settle
  } else {
    pinMode(HALL_PIN, INPUT);
    digitalWrite(HALL_PWR, LOW);
  }
}

float normDeg(float d) {
  d = fmodf(d, 360.0f);
  return d < 0 ? d + 360.0f : d;
}

// Angular difference a-b, in -180..180
float angleDiff(float a, float b) {
  float d = normDeg(a - b);
  return d > 180.0f ? d - 360.0f : d;
}

// Absolute step target for a bearing, taking the shortest way round from where we are
```

```
long dirToSteps(float dir) {
  long target = lround(normDeg(dir) * stepsPerDegree);
  long cur    = stepper.currentPosition();
  long curMod = ((cur % stepsPerRevolution) + stepsPerRevolution) % stepsPerRevolution;
  long delta  = target - curMod;
  if (delta >  stepsPerRevolution / 2) delta -= stepsPerRevolution;
  if (delta < -stepsPerRevolution / 2) delta += stepsPerRevolution;
  return cur + delta;
}

void applyNorthOffset() {
  stepper.setCurrentPosition(-lround(northOffsetDeg * stepsPerDegree));
}

// Start a move that always finishes travelling clockwise (takes up gear backlash)
long finalTarget = 0;
bool finalPending = false;

void moveToSteps(long target) {
  if (target < stepper.currentPosition()) {          // anticlockwise move
    stepper.moveTo(target - lround(backlashDeg * stepsPerDegree));   // overshoot
    finalTarget = target;
    finalPending = true;                             // then come back clockwise
  } else {
    stepper.moveTo(target);
    finalPending = false;
  }
}

// Called when a move ends: if it was the overshoot, start the clockwise return
bool continueMove() {
  if (!finalPending) return false;
  finalPending = false;
  stepper.moveTo(finalTarget);
  return true;
}

void runToPosition(long pos) {
  moveToSteps(pos);
  do {
    while (stepper.distanceToGo() != 0) stepper.run();
  } while (continueMove());
}

void powerCoils() {
  if (!coilsOn) { stepper.enableOutputs(); coilsOn = true; }
}

// ============================================================================
// HOMING & CALIBRATION
// ============================================================================
// Leaves the hall sensor powered — the caller switches it off when finished
void seekMagnetAndZero() {
  hallPower(true);
  powerCoils();
  stepper.setMaxSpeed(homingSpeed);
  stepper.setSpeed(homingSpeed);
  while (atMagnet())  stepper.runSpeed();   // step off if already on it
  while (!atMagnet()) stepper.runSpeed();   // roll on to the leading edge
  applyNorthOffset();
}

void calibrateStepsPerRevolution() {
  Serial.println("Calibrating: one magnet -> magnet revolution...");
  stepper.setCurrentPosition(0);
  stepper.setSpeed(homingSpeed);
  while (atMagnet())  stepper.runSpeed();
```

```
  while (!atMagnet()) stepper.runSpeed();
  long measured = labs(stepper.currentPosition());
  if (measured > 100) {
    stepsPerRevolution = measured;
    stepsPerDegree = stepsPerRevolution / 360.0;
  } else {
    Serial.println("Calibration looked wrong; keeping default.");
  }
  applyNorthOffset();
  Serial.printf("Steps/rev: %ld  steps/deg: %.4f\n", stepsPerRevolution, stepsPerDegree);
}

void rehome() {
  Serial.println("Auto re-home...");
  seekMagnetAndZero();
  hallPower(false);
  stepper.setMaxSpeed(runSpeed);
  stepper.setAcceleration(runAccel);
  runToPosition(0);          // park at North
  shownDir = NAN;            // force a fresh move to the current reading
}

// ============================================================================
// LEDS
// ============================================================================
void showBacklight() {
  const float k = LED_BRIGHTNESS / 255.0f;
  for (int i = 0; i < NUM_LEDS; i++)
    strip.setPixelColor(i, strip.Color(backlightRGB[0] * k, backlightRGB[1] * k, backlightRGB[2] * k));
  strip.show();
}

// Start-up progress shown on the whole ring (no Serial Monitor needed):
//   RED    = sketch has started
//   BLUE   = connecting to Wi-Fi
//   YELLOW = homing / calibrating (motor should be turning)
//   PURPLE = self-test sweep
//   WHITE  = ready (normal backlight)
void showStatus(uint8_t r, uint8_t g, uint8_t b) {
  strip.fill(strip.Color(r, g, b));
  strip.show();
}
#define STATUS_STARTED   showStatus(80, 0, 0)
#define STATUS_WIFI      showStatus(0, 0, 80)
#define STATUS_HOMING    showStatus(80, 60, 0)
#define STATUS_SELFTEST  showStatus(60, 0, 80)

// ============================================================================
// MQTT
// ============================================================================
float readNumber(JsonVariant v) {
  if (v.is<const char*>()) return atof(v.as<const char*>());
  if (v.is<float>() || v.is<long>()) return v.as<float>();
  return NAN;
}

float extractDirection(byte* payload, unsigned int length) {
  unsigned int i = 0;
  while (i < length && isspace(payload[i])) i++;
  if (i >= length) return NAN;

  // Plain number payload
  if (payload[i] != '{') {
    char buf[24]; unsigned int n = min(length - i, (unsigned int)sizeof(buf) - 1);
    memcpy(buf, payload + i, n); buf[n] = 0;
    if (!isdigit(buf[0]) && buf[0] != '-' && buf[0] != '.') return NAN;   // "None", "null"
    return atof(buf);
```

```
  }

  // JSON loop packet
  JsonDocument doc;
  if (deserializeJson(doc, (const char*)payload, length)) return NAN;
  const char* keys[] = { "windDir", "wind_dir", "windDir_degree_compass", "winddir" };
  for (const char* key : keys) {
    JsonVariant v = doc[key];
    if (v.isNull()) continue;            // weeWX sends null when it's calm
    float val = readNumber(v);
    if (!isnan(val)) return val;
  }
  return NAN;
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  float dir = extractDirection(payload, length);
  if (isnan(dir) || dir < -360 || dir > 720) return;   // calm or junk: hold position
  currentDir = normDeg(dir);

  static unsigned long lastPrint = 0;
  if (millis() - lastPrint > 60000) {
    lastPrint = millis();
    Serial.printf("Wind direction %.0f deg\n", currentDir);
  }
}

void connectWiFi() {
  Serial.printf("Connecting to Wi-Fi %s", ssid);
  WiFi.mode(WIFI_STA);
  // ESP32-C3 Super Mini: the tiny on-board antenna often won't connect at full
  // transmit power (and the current spike can reset the board). 8.5 dBm fixes it.
  WiFi.setTxPower(WIFI_POWER_8_5dBm);
  WiFi.setSleep(false);
  WiFi.setAutoReconnect(true);
  WiFi.begin(ssid, password);
  // Keep trying rather than rebooting (a reboot would re-home and re-sweep the pointer)
  for (int t = 1; WiFi.status() != WL_CONNECTED; t++) {
    delay(500); Serial.print(".");
    if (t % 40 == 0) {                       // every 20 s, start the attempt again
      Serial.println("\nStill trying Wi-Fi...");
      WiFi.disconnect();
      WiFi.setTxPower(WIFI_POWER_8_5dBm);
      WiFi.begin(ssid, password);
    }
  }
  Serial.printf("\nWi-Fi connected, IP %s\n", WiFi.localIP().toString().c_str());
}

void serviceNetwork() {
  if (WiFi.status() != WL_CONNECTED) return;     // ESP32 reconnects by itself
  if (!client.connected()) {
    static unsigned long lastTry = 0;
    if (millis() - lastTry < 5000) return;       // retry every 5 s, never block
    lastTry = millis();
    String id = "OpenGauge-Wind-" + String((uint32_t)ESP.getEfuseMac(), HEX);
    Serial.print("Connecting to MQTT... ");
    if (client.connect(id.c_str())) {
      Serial.println("connected");
      client.subscribe(wind_topic);
    } else {
      Serial.printf("failed rc=%d\n", client.state());
    }
    return;
  }
  client.loop();
}
```

```
// ============================================================================
// SETUP
// ============================================================================
void setup() {
  Serial.begin(115200);
  strip.begin();
  strip.setBrightness(255);           // full; backlight is dimmed per-pixel in showBacklight()
  STATUS_STARTED;
  delay(1500);
  Serial.println("\nOpen Gauges — Wind Direction");

  pinMode(HALL_PWR, OUTPUT);
  hallPower(false);                   // sensor off until it's needed

  // Wi-Fi first, before the motor runs, so the two big power draws never overlap
  STATUS_WIFI;
  connectWiFi();

  // Home, calibrate, park at North
  STATUS_HOMING;
  seekMagnetAndZero();
  delay(1000);
  calibrateStepsPerRevolution();
  hallPower(false);                   // homing done — sensor off
  stepper.setMaxSpeed(runSpeed);
  stepper.setAcceleration(runAccel);
  runToPosition(0);
  lastRehomeMs = millis();
  Serial.println("At North. Check the pointer against the N mark.");
  delay(2000);

  // Self-test: North -> South -> North
  Serial.println("Self-test: N -> S -> N");
  STATUS_SELFTEST;
  runToPosition(dirToSteps(180));  delay(1000);
  runToPosition(dirToSteps(0));

  client.setServer(mqtt_server, mqtt_port);
  client.setBufferSize(4096);         // weeWX loop packets are larger than the 256-byte default
  client.setCallback(mqttCallback);

  showBacklight();                    // normal white backlight = ready
  Serial.println("Ready.");
}

// ============================================================================
// LOOP — while the pointer moves, do nothing but step (smooth motion)
// ============================================================================
void loop() {
  if (stepper.distanceToGo() != 0) {
    stepper.run();
    return;
  }
  if (continueMove()) return;     // finish an anticlockwise move from the clockwise side

  if (coilsOn && releaseCoilsWhenIdle) { stepper.disableOutputs(); coilsOn = false; }

  static unsigned long lastNet = 0;
  if (millis() - lastNet >= 50) { lastNet = millis(); serviceNetwork(); }

  if (millis() - lastRehomeMs > rehomeIntervalMs) {
    rehome();
    lastRehomeMs = millis();
  }

  // Move the pointer if the reading has changed enough
```

```
  if (!isnan(currentDir) && (isnan(shownDir) || fabsf(angleDiff(currentDir, shownDir)) >= moveDeadbandDeg)) {
    powerCoils();
    moveToSteps(dirToSteps(currentDir));
    shownDir = currentDir;
  }

  stepper.run();
}
```